

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**Applied Business Analytics Practical**

## Practical – X

|                                  |                                     |
|----------------------------------|-------------------------------------|
| Roll No.: T009                   | Name: Siddhesh Kamble               |
| Class: TYIT                      | Batch: 01                           |
| Date of Assignment: : 03/10/2026 | Date/Time of Submission: 03/10/2026 |

### Practical 10: Regression Analysis and Time Series Analysis

#### Part A: Regression Analysis

**Dataset:** sales\_data.csv

1. Load the sales\_data.csv dataset.
2. Display the first five records.
3. Identify the independent and dependent variables.
4. Select advertising as the independent variable and sales as the dependent variable.
5. Check the data for missing values.
6. Create a scatter plot of advertising expenditure and sales.
7. Develop a simple linear regression model.
8. Display the regression equation.
9. Use the model to predict sales for a given advertising expenditure.
10. Calculate the R2 score of the model.
11. Calculate the Mean Squared Error (MSE).
12. Compare the actual sales with the predicted sales.
13. Plot the regression line with the actual data points.
14. Interpret the regression results.
15. Write a conclusion about how advertising expenditure is related to sales.

**A simple regression model is:**

**Sales = b0 + b1 (Advertising)**

**Code :**

```
# Practical 10: Regression Analysis
# Part A: Regression Analysis

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# 1. Load the sales_data.csv dataset
df = pd.read_csv("sales_data.csv")
# 2. Display the first five records
print("First 5 Records:")
print(df.head())
# 3. Identify the independent and dependent variables
print("\nIndependent Variable: advertising")
print("Dependent Variable: sales")
# 4. Select advertising as the independent variable and sales as the dependent variable
X = df[["advertising"]]
y = df["sales"]
# 5. Check the data for missing values
print("\nMissing Values:")
print(df.isnull().sum())
# 6. Create a scatter plot of advertising expenditure and sales
plt.scatter(df["advertising"], df["sales"])
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
```

```

plt.title("Advertising Expenditure vs Sales")
plt.show()
# 7. Develop a simple linear regression model
model = LinearRegression()
model.fit(X, y)
# 8. Display the regression equation
b0 = model.intercept_
b1 = model.coef_[0]
print("\nRegression Equation:")
print(f"Sales = {b0:.2f} + {b1:.2f} (Advertising)")
# 9. Use the model to predict sales for a given advertising expenditure
advertising_value = 50
predicted_sales = model.predict([[advertising_value]])
print(f"\nPredicted Sales for Advertising = {advertising_value}: "
      f"{predicted_sales[0]:.2f}")

# Predicted sales for all records
y_pred = model.predict(X)
# 10. Calculate the R2 score of the model
r2 = r2_score(y, y_pred)
print(f"\nR2 Score: {r2:.4f}")
# 11. Calculate the Mean Squared Error (MSE)
mse = mean_squared_error(y, y_pred)
print(f"Mean Squared Error (MSE): {mse:.4f}")
# 12. Compare the actual sales with the predicted sales
comparison = pd.DataFrame({
    "Actual Sales": y,
    "Predicted Sales": y_pred
})
print("\nActual vs Predicted Sales:")
print(comparison)
# 13. Plot the regression line with the actual data points
plt.scatter(X, y, label="Actual Data")
plt.plot(X, y_pred, label="Regression Line")
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
plt.title("Linear Regression: Advertising vs Sales")
plt.legend()
plt.show()
# 14. Interpret the regression results
print("\nInterpretation:")
print("The regression model shows the relationship between advertising expenditure and sales.")
print(f"For every one-unit increase in advertising, sales change by approximately {b1:.2f} units.")
# 15. Write a conclusion
print("\nConclusion:")
print("Advertising expenditure has a positive relationship with sales.")
print("Therefore, increasing advertising expenditure is associated with an increase in sales.")

```

**Output :**

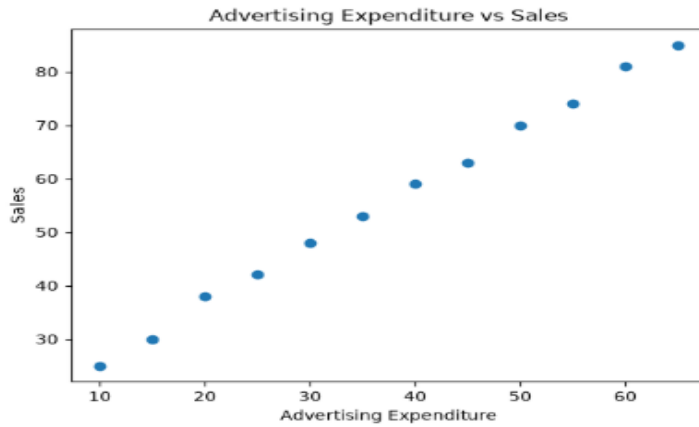
```

... First 5 Records:
   advertising  sales
0            10     25
1            15     30
2            20     38
3            25     42
4            30     48

Independent Variable: advertising
Dependent Variable: sales

Missing Values:
advertising    0
sales         0
dtype: int64

```



Regression Equation:  
 $\text{Sales} = 14.71 + 1.09 (\text{Advertising})$

Predicted Sales for Advertising = 50: 69.32

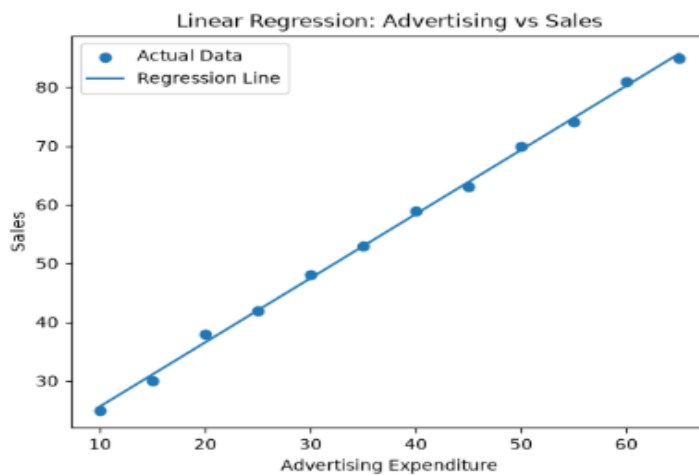
R2 Score: 0.9983

Mean Squared Error (MSE): 0.6004

Actual vs Predicted Sales:

|    | Actual Sales | Predicted Sales |
|----|--------------|-----------------|
| 0  | 25           | 25.628205       |
| 1  | 30           | 31.089744       |
| 2  | 38           | 36.551282       |
| 3  | 42           | 42.012821       |
| 4  | 48           | 47.474359       |
| 5  | 53           | 52.935897       |
| 6  | 59           | 58.397436       |
| 7  | 63           | 63.858974       |
| 8  | 70           | 69.320513       |
| 9  | 74           | 74.782051       |
| 10 | 81           | 80.243590       |
| 11 | 85           | 85.705128       |

<c:\Users\mvluc\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\utils\validation.py:2827>: UserWarning  
 warnings.warn()



Interpretation:

The regression model shows the relationship between advertising expenditure and sales. For every one-unit increase in advertising, sales change by approximately 1.09 units.

Conclusion:

Advertising expenditure has a positive relationship with sales. Therefore, increasing advertising expenditure is associated with an increase in sales.

**Part B: Time Series Analysis****Dataset: monthly\_sales.csv****The dataset can contain:**

| Month | Sales |
|-------|-------|
| Jan   | 120   |
| Feb   | 135   |
| Mar   | 128   |
| Apr   | 145   |
| May   | 150   |
| Jun   | 160   |
| Jul   | 155   |
| Aug   | 170   |
| Sep   | 165   |
| Oct   | 180   |
| Nov   | 190   |
| Dec   | 200   |

**Steps:**

16. Load monthly sales data.
17. Convert the date column into date format.
18. Plot monthly sales using a line chart.
19. Observe the trend.
20. Apply Exponential Smoothing.
21. Forecast the next 3 months.
22. Plot actual and forecasted sales.
23. Calculate forecasting error.
24. Apply ARIMA to the same data.
25. Forecast the next 3 months using ARIMA.
26. Compare the ARIMA and Exponential Smoothing forecasts.
27. Calculate and compare their forecasting errors.
28. Identify which model gives the lower forecasting error for the given dataset.
29. Write the final conclusion.

**Code :**

```
# Practical 10: Time Series Analysis
# Part B: Time Series Analysis
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
# 16. Load monthly sales data
df = pd.read_csv("monthly_sales.csv")
print("Monthly Sales Data:")
print(df)
# 17. Convert the date column into date format
df["Date"] = pd.to_datetime(
```

```
        df["Month"] + "-2026",
        format="%b-%Y"
    )
    df.set_index("Date", inplace=True)
    print("\nData after converting Month into Date:")
    print(df)
    # 18. Plot monthly sales using a line chart
    plt.figure(figsize=(10, 5))
    plt.plot(df.index, df["Sales"], marker="o")
    plt.xlabel("Month")
    plt.ylabel("Sales")
    plt.title("Monthly Sales")
    plt.grid(True)
    plt.show()
    # 19. Observe the trend
    print("\nTrend:")
    print("The monthly sales show an overall increasing trend.")
    # 20. Apply Exponential Smoothing
    es_model = ExponentialSmoothing(
        df["Sales"],
        trend="add",
        seasonal=None
    ).fit()
    # 21. Forecast the next 3 months
    es_forecast = es_model.forecast(3)
    print("\nExponential Smoothing Forecast:")
    print(es_forecast)
    # 22. Plot actual and forecasted sales
    plt.figure(figsize=(10, 5))
    plt.plot(
        df.index,
        df["Sales"],
        marker="o",
        label="Actual Sales"
    )
    future_dates = pd.date_range(
        start=df.index[-1] + pd.DateOffset(months=1),
        periods=3,
        freq="MS"
    )
    plt.plot(
        future_dates,
        es_forecast,
        marker="o",
        label="Exponential Smoothing Forecast"
    )
    plt.xlabel("Month")
    plt.ylabel("Sales")
    plt.title("Actual and Exponential Smoothing Forecast")
    plt.legend()
    plt.grid(True)
    plt.show()
    # 23. Calculate forecasting error
    # Calculate fitted values and error
    es_fitted = es_model.fittedvalues
    es_mse = mean_squared_error(df["Sales"], es_fitted)
    print(f"\nExponential Smoothing MSE: {es_mse:.4f}")
    # 24. Apply ARIMA to the same data
    arima_model = ARIMA(
        df["Sales"],
        order=(1, 1, 1)
    ).fit()
    # 25. Forecast the next 3 months using ARIMA
    arima_forecast = arima_model.forecast(steps=3)
    print("\nARIMA Forecast:")
    print(arima_forecast)
    # 26. Compare the ARIMA and Exponential Smoothing forecasts
```

```

forecast_comparison = pd.DataFrame({
    "Exponential Smoothing": es_forecast.values,
    "ARIMA": arima_forecast.values
}, index=future_dates)
print("\nForecast Comparison:")
print(forecast_comparison)
# 27. Calculate and compare their forecasting errors
arima_fitted = arima_model.fittedvalues
arima_mse = mean_squared_error(
    df["Sales"].iloc[1:],
    arima_fitted.iloc[1:]
)
print("\nForecasting Errors:")
print(f"Exponential Smoothing MSE: {es_mse:.4f}")
print(f"ARIMA MSE: {arima_mse:.4f}")
# 28. Identify which model gives the lower forecasting error
if es_mse < arima_mse:
    print("\nModel with Lower Forecasting Error: Exponential Smoothing")
elif arima_mse < es_mse:
    print("\nModel with Lower Forecasting Error: ARIMA")
else:
    print("\nBoth models have the same forecasting error.")
# Comparison chart
plt.figure(figsize=(10, 5))
plt.plot(
    df.index,
    df["Sales"],
    marker="o",
    label="Actual Sales"
)
plt.plot(
    future_dates,
    es_forecast,
    marker="o",
    label="Exponential Smoothing"
)
plt.plot(
    future_dates,
    arima_forecast,
    marker="o",
    label="ARIMA"
)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("ARIMA vs Exponential Smoothing Forecast")
plt.legend()
plt.grid(True)
plt.show()
# 29. Write the final conclusion
print("\nFinal Conclusion:")
print("The monthly sales data shows an overall increasing trend.")
print("Both Exponential Smoothing and ARIMA are used to forecast the next three months.")
print("The model with the lower MSE provides the lower forecasting error for this dataset.")

```

**Output :**

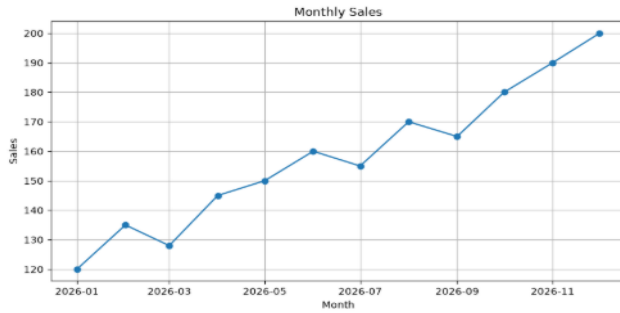
```

Monthly Sales Data:
  Month  Sales
0   Jan    120
1   Feb    135
2   Mar    128
3   Apr    145
4   May    150
5   Jun    160
6   Jul    155
7   Aug    170
8   Sep    165
9   Oct    180
10  Nov    190
11  Dec    200

Data after converting Month into Date:
   Date          Month  Sales
0 2020-01-01      Jan    120
1 2020-02-01      Feb    135
2 2020-03-01      Mar    128
3 2020-04-01      Apr    145
4 2020-05-01      May    150
5 2020-06-01      Jun    160
6 2020-07-01      Jul    155
...
8 2020-09-01      Sep    165
9 2020-10-01     Oct    180
10 2020-11-01     Nov    190
11 2020-12-01     Dec    200

```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.



Trend:  
The monthly sales show an overall increasing trend.

Exponential Smoothing Forecast:

2027-01-01 201.212121

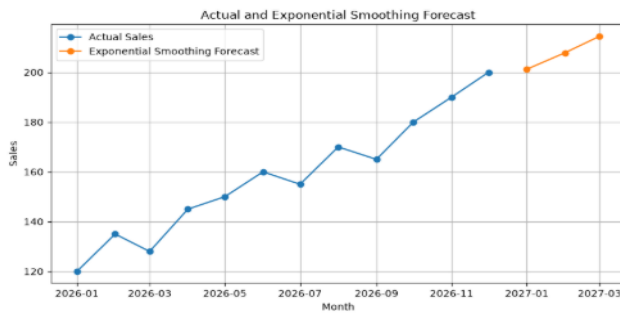
2027-02-01 207.834499

2027-03-01 214.456876

Freq: MS, dtype: float64

[c:\Users\mvjuc\AppData\Local\Programs\Python\Python314\lib\site-packages\statsmodels\tsa\base\tsa\\_model.py:473](#): ValueWarning: No frequency information was provided, so inferred frequency MS will be used.

self.\_init\_dates(dates, freq)



Exponential Smoothing MSE: 26.0229

[c:\Users\mvjuc\AppData\Local\Programs\Python\Python314\lib\site-packages\statsmodels\tsa\base\tsa\\_model.py:473](#): ValueWarning: No frequency information was provided, so inferred frequency MS will be used.

self.\_init\_dates(dates, freq)

[c:\Users\mvjuc\AppData\Local\Programs\Python\Python314\lib\site-packages\statsmodels\tsa\base\tsa\\_model.py:473](#): ValueWarning: No frequency information was provided, so inferred frequency MS will be used.

self.\_init\_dates(dates, freq)

[c:\Users\mvjuc\AppData\Local\Programs\Python\Python314\lib\site-packages\statsmodels\tsa\base\tsa\\_model.py:473](#): ValueWarning: No frequency information was provided, so inferred frequency MS will be used.

self.\_init\_dates(dates, freq)

ARIMA Forecast:

2027-01-01 193.259353

2027-02-01 199.999922

2027-03-01 193.259431

Freq: MS, Name: predicted\_mean, dtype: float64

Forecast Comparison:

|            | Exponential Smoothing | ARIMA      |
|------------|-----------------------|------------|
| 2027-01-01 | 201.212121            | 193.259353 |
| 2027-02-01 | 207.834499            | 199.999922 |
| 2027-03-01 | 214.456876            | 193.259431 |

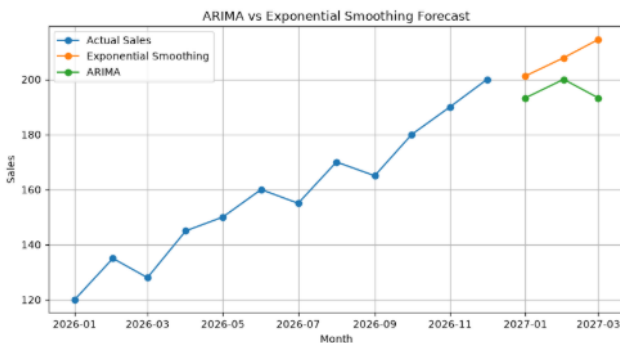
Forecasting Errors:

Exponential Smoothing MSE: 26.0229

ARIMA MSE: 92.7650

Model with Lower Forecasting Error: Exponential Smoothing

[c:\Users\mvjuc\AppData\Local\Programs\Python\Python314\lib\site-packages\statsmodels\tsa\base\tsa\\_model.py:687](#): ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle\_retvals warnings.warn("Maximum Likelihood optimization failed to "



Final Conclusion:

The monthly sales data shows an overall increasing trend.

Both Exponential Smoothing and ARIMA are used to forecast the next three months.

The model with the lower MSE provides the lower forecasting error for this dataset.